Suleyman Demirel University

CSS 217 Software Architecture and Design Patterns

Hotel Reservation System

Assignment: Strategy, Adapter and Decorator patterns

Aida Akmurat

Bagdat Atabek

Erasyl Kurmanbek

Almaty, 2026

A. Requirements Specification

Problem and goals

In many small hotels bookings are still written in a notebook. Because of this two guests can get the same room, prices are calculated by hand and payments get lost. Our program solves this: customers find a free room, book it and pay, and the administrator manages rooms, prices and check-in/check-out. Everything is saved in files.

Our goals were: no double bookings, automatic prices, optional extra services, simulated card payment, and a clean layered structure where the three design patterns are used naturally.

Users

- **Customer** – registers, searches rooms, books, pays, cancels, reads notifications.
- **Administrator** – manages rooms and prices, sees all bookings and payments, does check-in and check-out, chooses the pricing strategy.
- **QuickPay** – external (simulated) payment system.

Functional requirements

1. Show all rooms with type, capacity and price per night.
2. Find rooms that are free for the chosen dates and number of guests.
3. Create a reservation. Past dates, wrong date order, too many guests and already booked rooms are not allowed.
4. Calculate the price by room type, season and length of stay, using one of three pricing strategies.
5. Let the administrator change the pricing strategy while the program is running.
6. Let the customer add breakfast, airport transfer and late check-out.
7. Pay by card. After payment the reservation becomes CONFIRMED; failed payments are also saved.
8. Cancel a reservation.
9. Save notifications about every important action.
10. Check-in (only paid bookings, from the check-in date) and check-out.
11. Registration and login with two roles.
12. Keep all data after the program is closed.

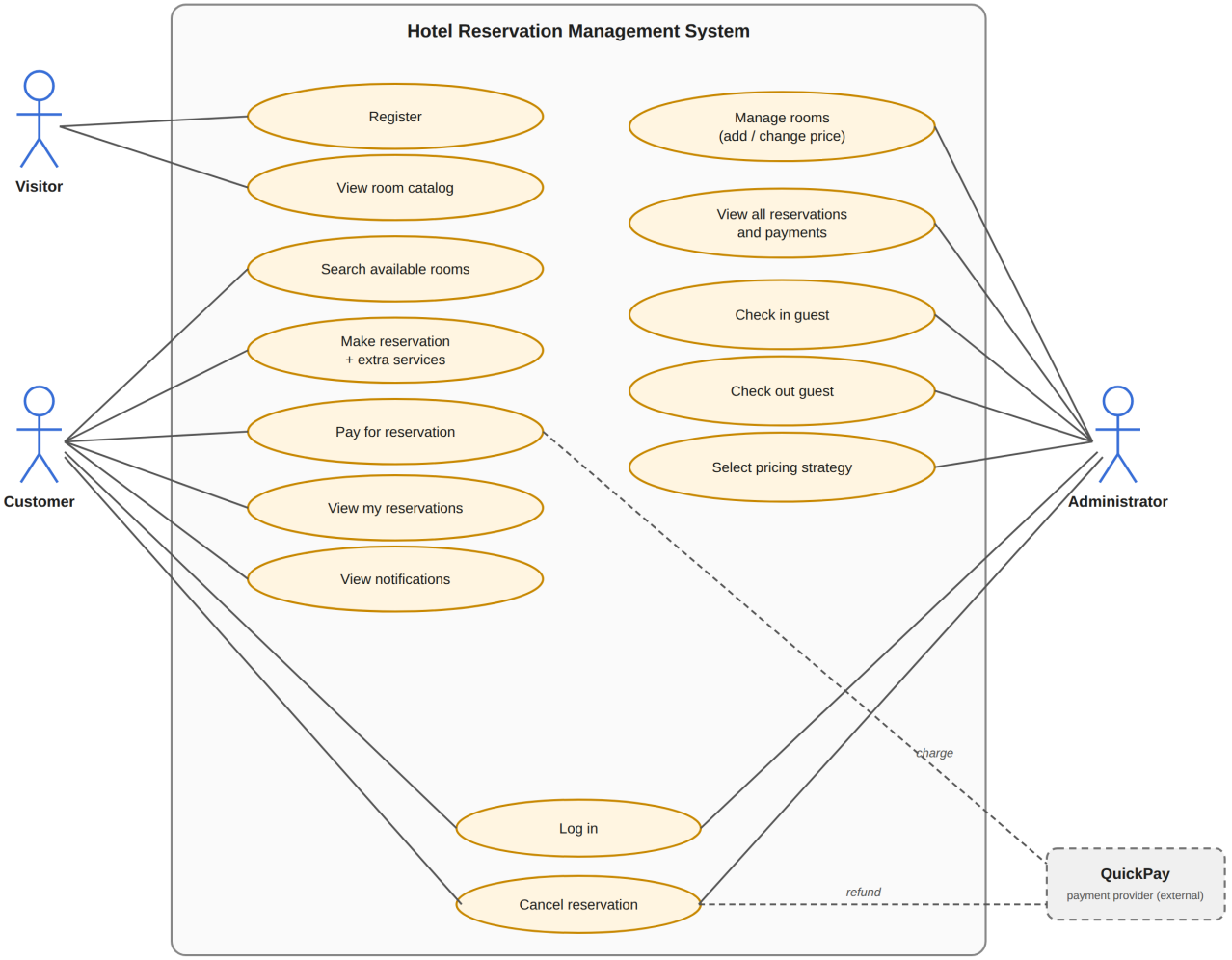
Non-functional requirements

- Works with only JDK 17 or newer, no extra libraries.
- Wrong input does not stop the program, it shows a message and asks again.
- Passwords are saved as hashes, card numbers are not saved.
- New strategies, extra services or payment systems can be added without changing the existing services.

Use cases

A customer makes a reservation like this: enter dates and guests → choose a room → answer y/n for each extra service → see the total → confirm → pay now or later. The reservation goes through these states: PENDING_PAYMENT → CONFIRMED → CHECKED_IN → CHECKED_OUT, or it can be CANCELLED before check-in.

Use-case diagram

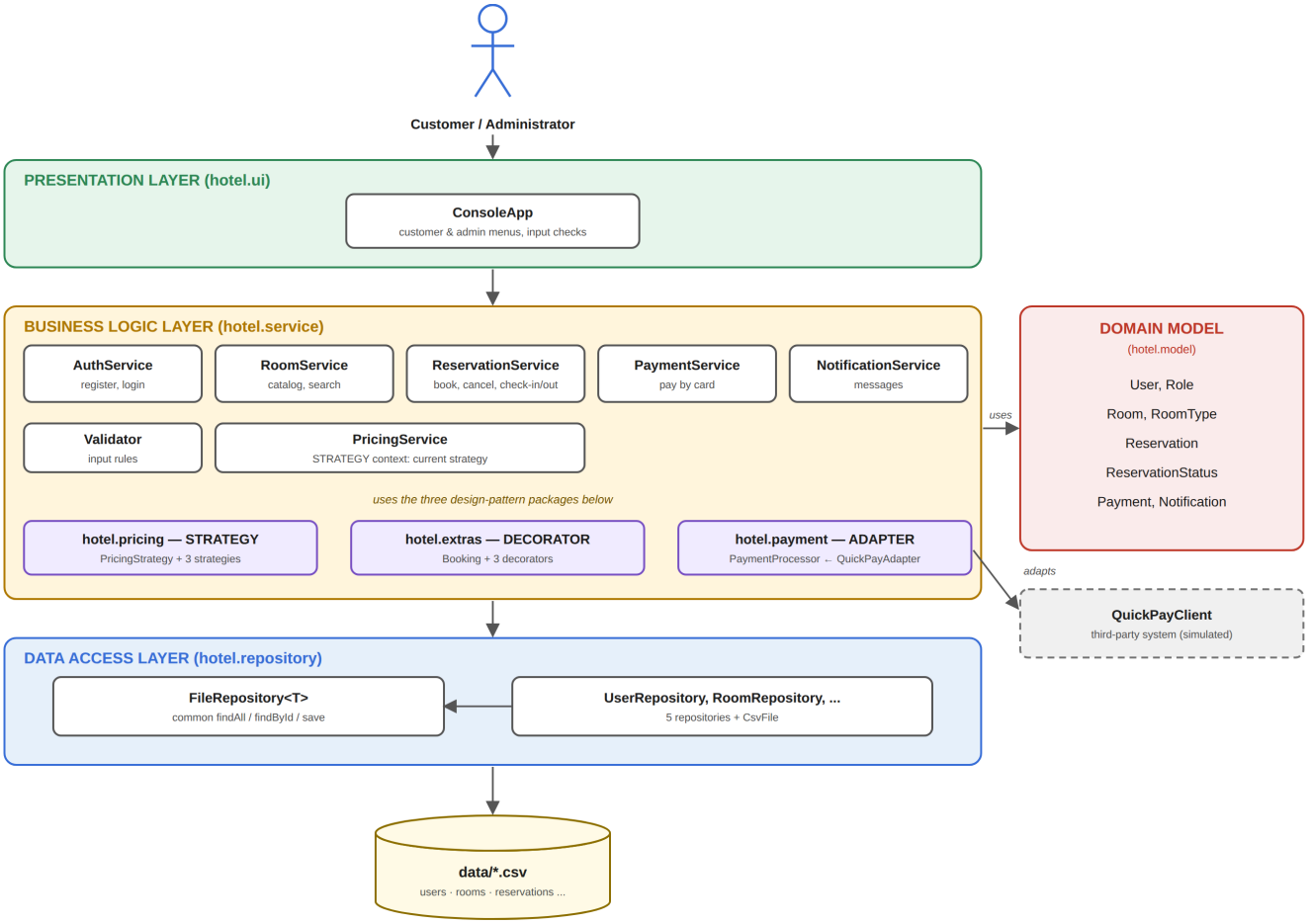


B. Software Design

Architecture

The program has three layers: **presentation** (console menu), **business logic** (services) and **data access** (repositories that read and write CSV files). Each layer only calls the layer below it. All layers use the same model classes.

Architecture diagram



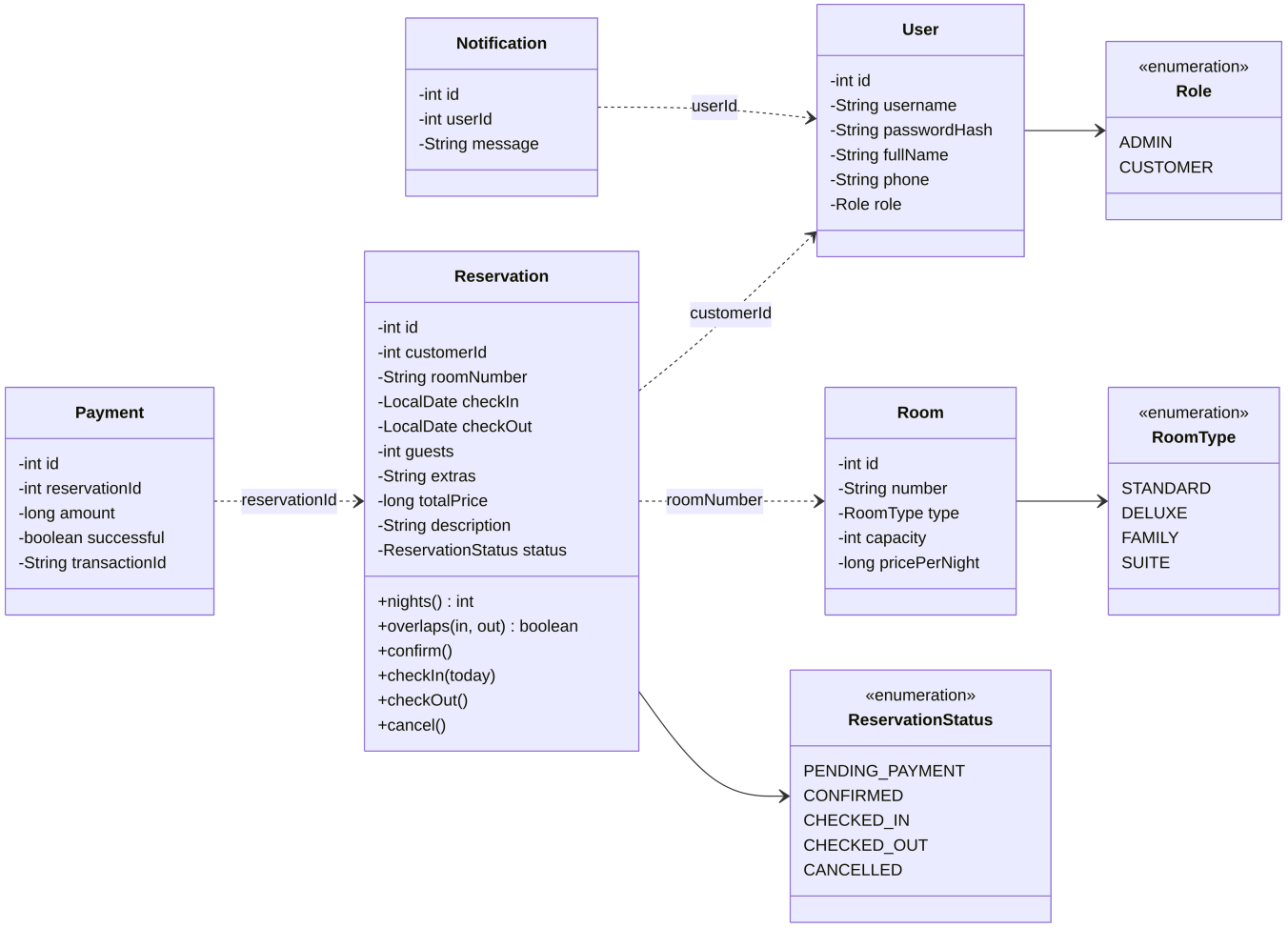
Files

Package	Files	What they do
`ui`	ConsoleApp	menus and reading input
`service`	AuthService, RoomService, ReservationService, PaymentService, PricingService, NotificationService, Validator	rules of the hotel
`repository`	CsvFile, FileRepository, UserRepository, RoomRepository, ReservationRepository, PaymentRepository, NotificationRepository	saving and loading data
`model`	User, Room, Reservation, Payment, Notification, Role, RoomType, ReservationStatus	main objects
`exception`	HotelException, ValidationException, NotFoundException, PaymentException	errors shown to the user
`pricing`, `extras`, `payment`	see the patterns below	
–	Main	starts the program and connects everything

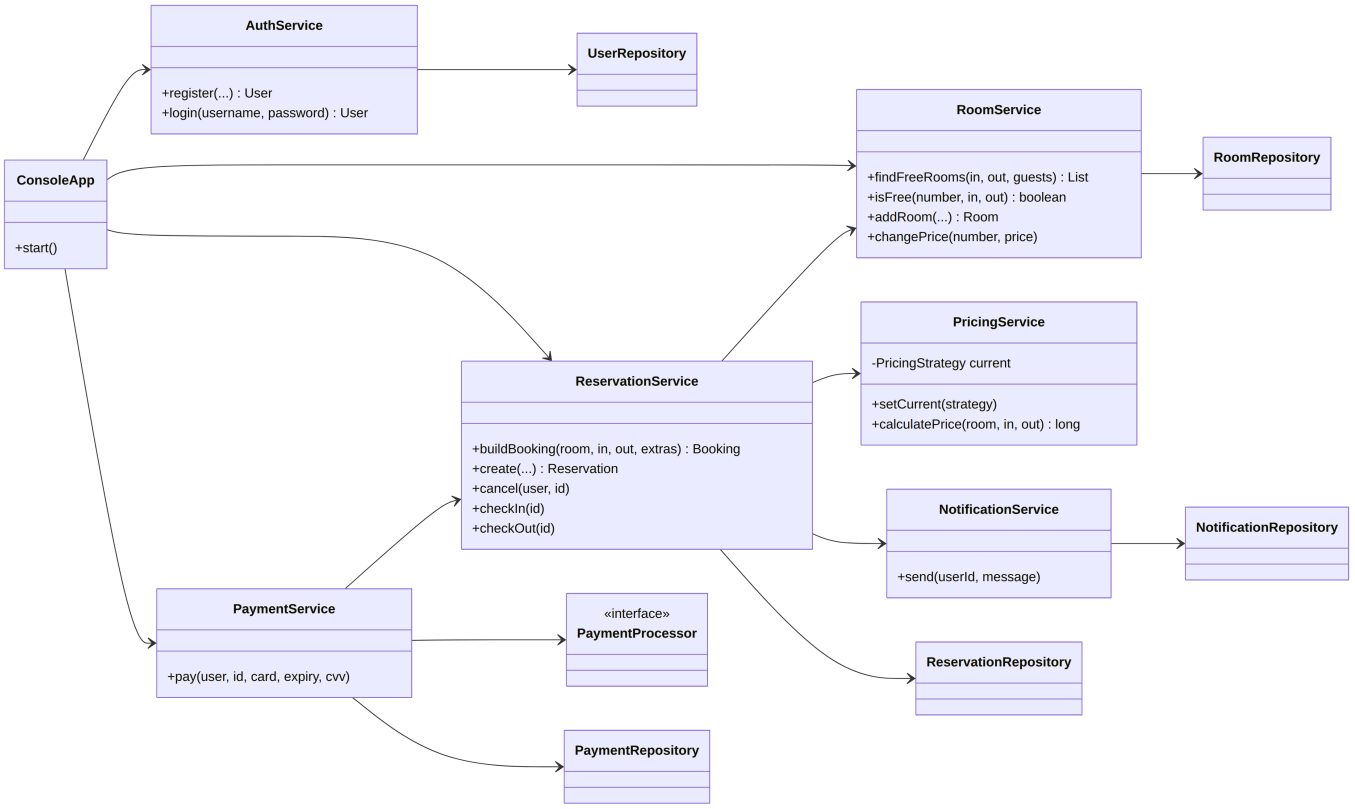
UML class diagram

The diagram is split into two parts: the model classes and the services with repositories.

Class diagram – part 1: model



Class diagram – part 2: services and repositories



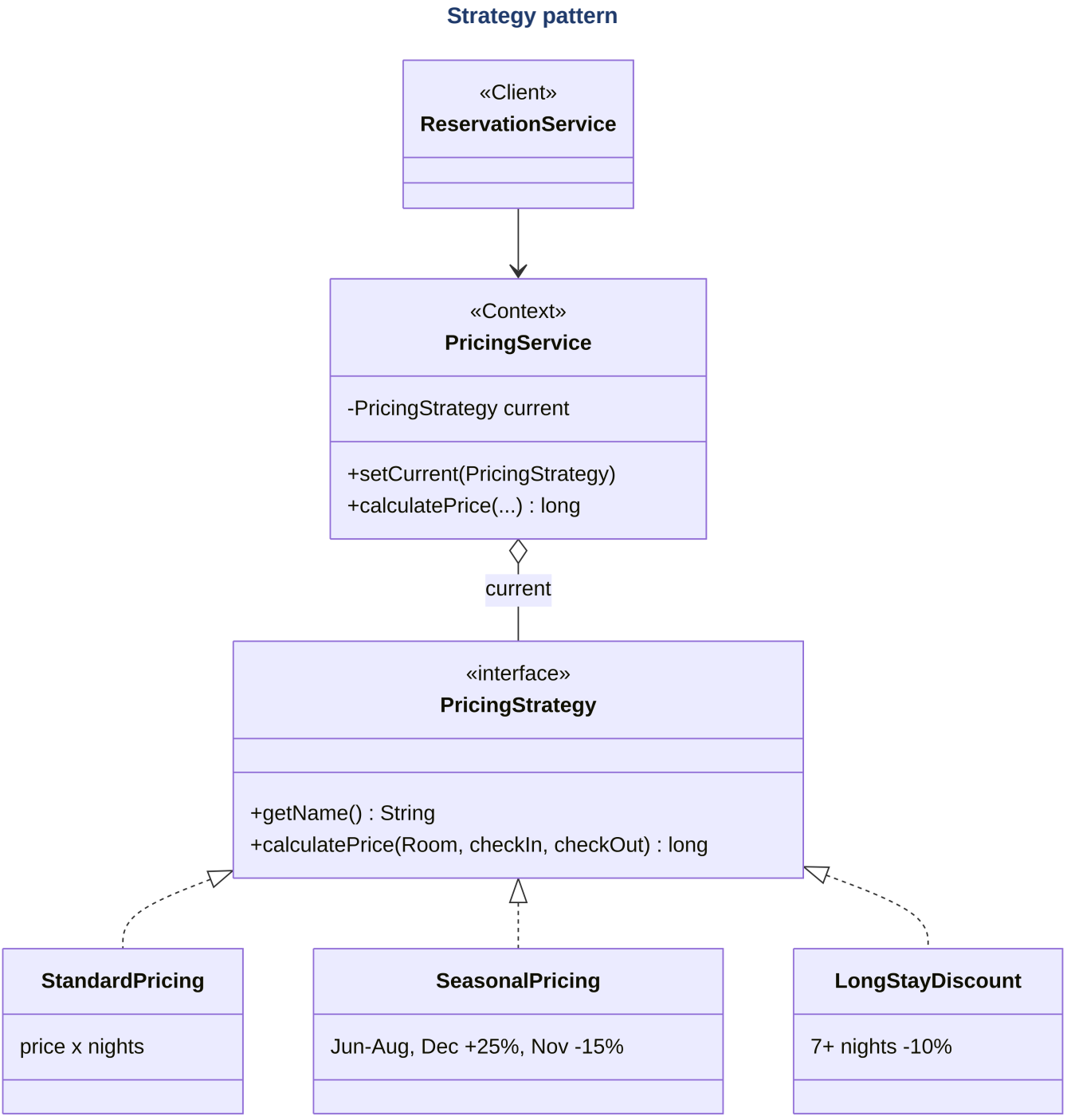
Design patterns

★ Strategy – price calculation

The price can be calculated in different ways, so each way is a separate class with the same interface `PricingStrategy`:

- `StandardPricing` – price × nights
- `SeasonalPricing` – summer and December +25%, November −15%
- `LongStayDiscount` – 7+ nights −10%

`PricingService` keeps the current strategy, and the administrator can change it from the menu at any time. `ReservationService` just asks for the price and does not know which strategy is used. To add a new strategy we only write a new class and add it to the list in `Main`.

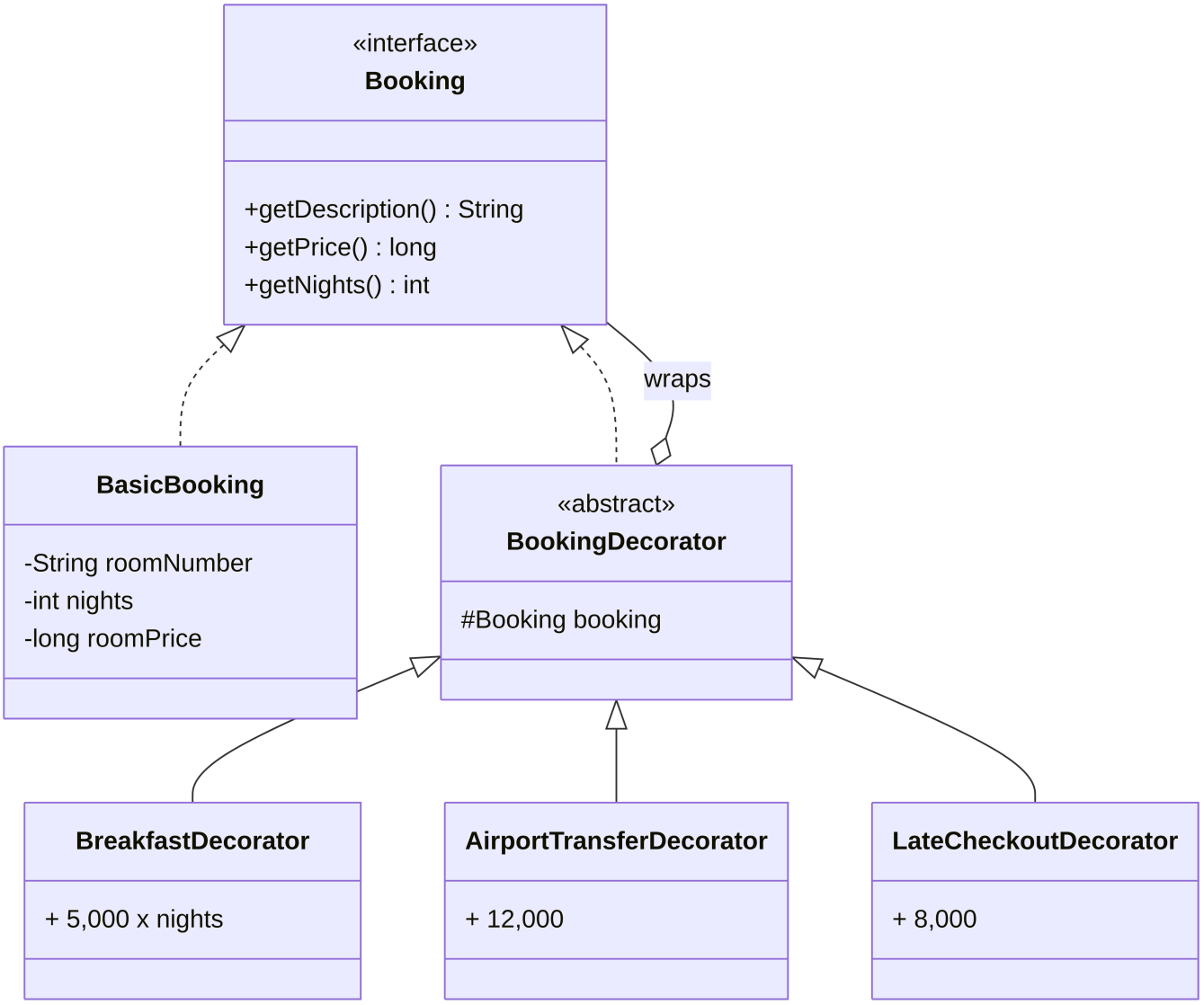


★ Decorator – extra services

`BasicBooking` is a room without extras. `BreakfastDecorator`, `AirportTransferDecorator` and `LateCheckoutDecorator` wrap a booking and add their price and name, so any combination is possible without a separate class for every combination:

```
        Booking b = new LateCheckoutDecorator(new BreakfastDecorator(new BasicBooking("201", 3,
120000)));
```

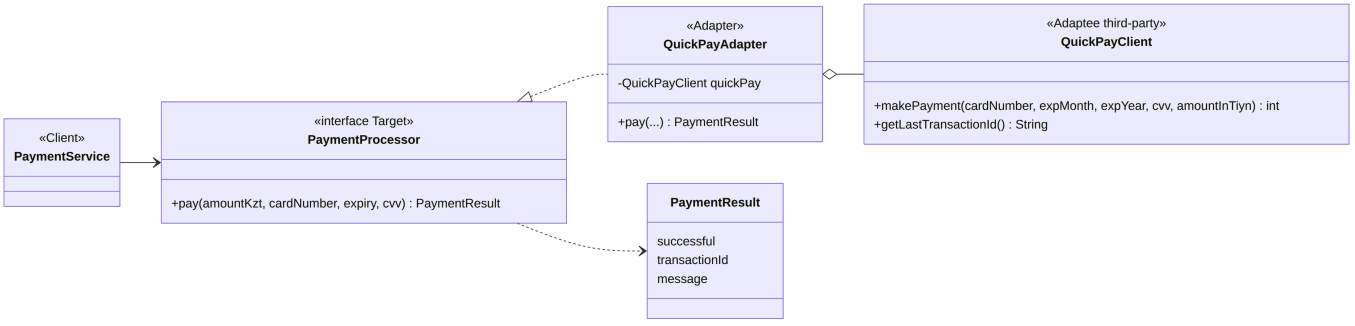
Decorator pattern



★ Adapter – payment

`QuickPayClient` plays the role of an external payment system that we cannot change. It wants the amount in tiyn and the expiry month and year as separate numbers, and it answers with a number code. Our program uses the simpler interface `PaymentProcessor`. `QuickPayAdapter` connects them: it converts KZT to tiyn, splits “MM/YY” and turns codes like 51 into messages like “Insufficient funds on the card”.

Adapter pattern



Main design decisions

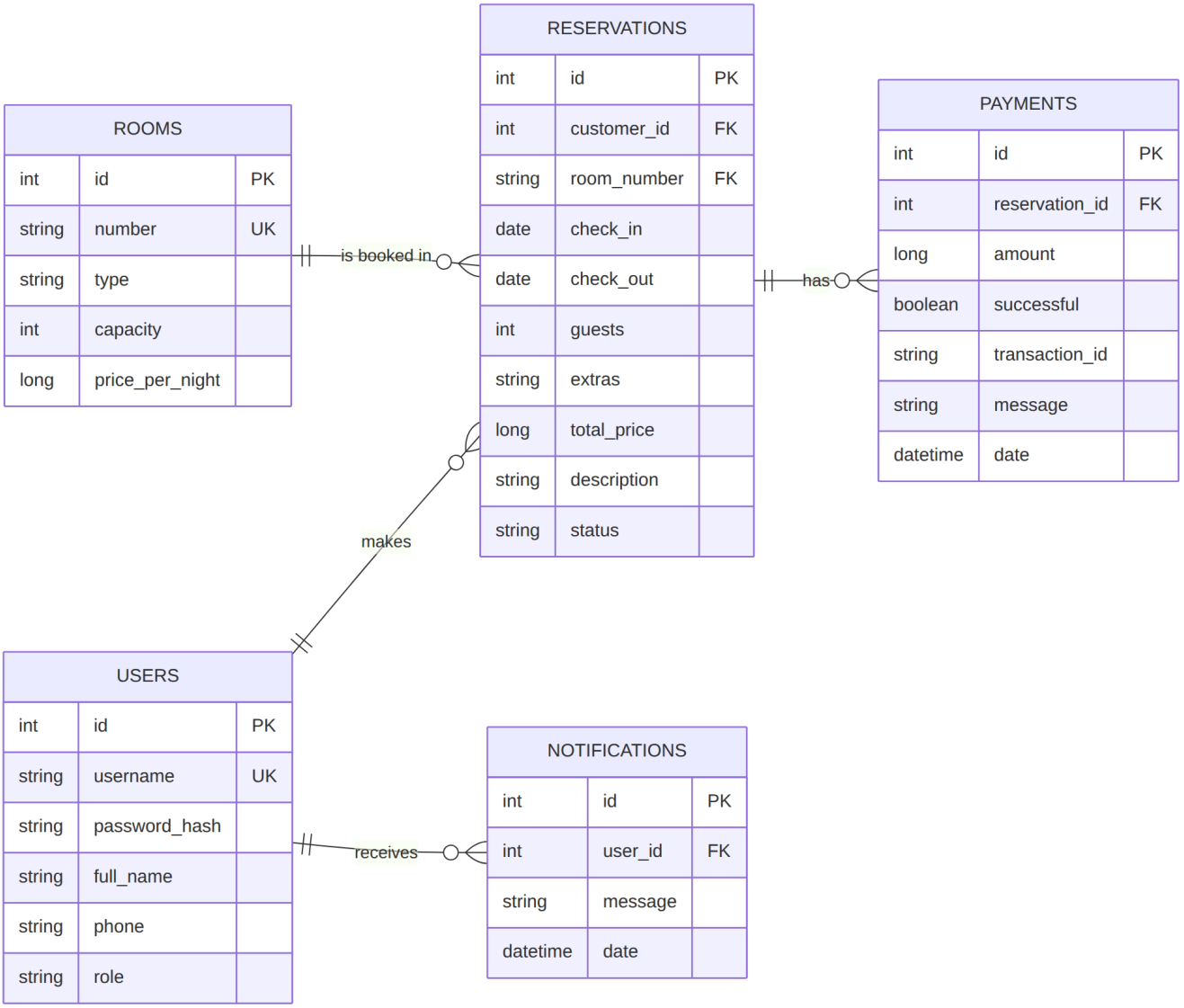
- Rules about status (for example, no check-in without payment) are inside the `Reservation` class, so they always work.
- Input is checked in the services, not only in the menu.
- All expected errors extend `HotelException`; the menu catches them and continues.
- A reservation saves its final price, so later price changes do not affect old bookings.

C. Database Design

Data is stored in the `data` folder as CSV files with `;` between values. Each file is one table and the first line has the column names. On the first start the sample data is copied from `database/seed`.

A user can have many reservations and notifications, a room can have many reservations (but not on the same nights), and a reservation can have many payment attempts. The ER diagram shows all columns and keys.

ER diagram



How it works: `CsvFile` reads and writes the lines. `FileRepository` loads all records, gives a new record the next id or replaces an old one, and writes the file again. Each repository only knows how to turn its object into a line and back. Unique usernames and free rooms are checked in the services.

D. Testing

There are 16 automated tests in `test/hotel/HotelTests.java`. Each test uses a new empty folder, so real data is not touched.

Test	What is checked	Result
T1	register and login, password is hashed	PASS
T2	new reservation: status and price (135,000)	PASS
T3	overlapping booking is rejected	PASS
T4	payment → CONFIRMED	PASS
T5	cancel, only by the owner, room becomes free	PASS
T6	check-in only after payment, then check-out	PASS
T7	data is read back from files	PASS
S1	3 strategies give 120,000 / 118,000 / 120,000 (7 nights: 252,000)	PASS
S2	strategy changed at runtime, a new strategy works without code changes	PASS
D1	3 decorators together: 155,000	PASS
D2	original booking is not changed	PASS
A1	adapter sends 1,500 KZT as 150,000 tiyn	PASS
A2	error codes become messages, unpaid booking stays unpaid	PASS
E1	wrong and past dates	PASS
E2	wrong password, taken username, short password or phone	PASS
E3	too many guests, unknown room, wrong room number	PASS

Result: **16 passed, 0 failed.** We also checked the program by hand with wrong logins, wrong dates, a declined card, and check-in before the check-in date. Each time the program showed a message and continued.

E. User Guide

Run: you need JDK 17+. Open a terminal in the project folder and type `./run.sh` (Windows: `run.bat`). Use `./run.sh --reset` to restore the sample data and `./test.sh` to run the tests. In IntelliJ, mark `src` as Sources Root and run `hotel.Main`.

Logins: admin / admin123, aida / aida123, bagdat / bagdat123. **Test cards:** 4242 4242 4242 4242 works, 4000 0000 0000 0002 has no money. Expiry 12/30, CVV 123.

Menus: the customer can find rooms, book, pay, cancel and read notifications. The administrator can view and add rooms, change prices, see bookings and payments, check guests in and out, and choose the pricing strategy. Dates are written as YYYY-MM-DD.

Check-in works only from the check-in date, so to show it, book a room starting today.